

Structure de Types de Données Abstraites

Exercices

1. Exercice 1 : Choisir un TAD selon la situation

Q1. Choisir le type abstrait de données adéquat pour chacune des tâches suivantes :

- ➔ Stocker un répertoire téléphonique
- ➔ Stocker l'historique de navigation d'un navigateur web
- ➔ Stocker les fichiers à imprimer dans un serveur d'impression
- ➔ Stocker les cartes d'un joueur dans le jeu de la bataille

2. Exercice 2 : Utiliser l'interface d'un TAD

Q1. Pour chaque séquence de primitives suivantes, **représenter** par un croquis l'état du TAD à chaque étape

Séquence 1

```
l = creer_liste()
insérer(l, 'A', 1)
insérer(l, 'O', 2)
insérer(l, 'B', 1)
insérer(l, 'V', 3)
insérer(l, 'R', 2)
```

Séquence 3

```
l1 = creer_liste()
l2 = creer_liste()
insérer(l1, 1, 1)
insérer(l1, 2, 2)
insérer(l1, 3, 3)
insérer(l1, 4, 4)
insérer(l2, lire(l1, 1), 1)
insérer(l2, lire(l1, 2), 1)
insérer(l2, lire(l1, 3), 1)
insérer(l2, lire(l1, 4), 1)
```

Séquence 2

```
f = creer_file()
enfiler(f, 4)
enfiler(f, 1)
enfiler(f, 3)
n = defiler(f)
enfiler(f, 8)
n = defiler(f)
```

Séquence 4

```
p = creer_pile()
empiler(p, 4)
empiler(p, 3)
n = depiler(p)
empiler(p, 8)
n = depiler(p)
```



3. Exercice 3 : Reasonner avec un TAD

On souhaite développer un nouveau TAD appelé *2extremities* ayant la capacité d'insérer ou de retirer un élément aux deux extrémités d'une pile. L'interface est le suivant :

Primitives	Opérations
<code>creer_2extremities()</code>	Retourne un « <i>2extremities</i> » vide
<code>2_est_vide(t)</code>	Teste si t est vide et renvoie True dans ce cas.
<code>insérer_gauche(t,e)</code>	Insère à gauche de t un élément e
<code>retirer_gauche(t)</code>	Retire et renvoie l'élément situé à gauche de t
<code>insérer_droite(t,e)</code>	Insère à droite de t un élément e
<code>retirer_droite(t)</code>	Retire et renvoie l'élément situé à droite de t

Cette nouvelle structure sera implémentée avec une pile principale associée à une pile secondaire qui permettra de renverser la première si besoin. La primitive `insérer_droite(t,e)` revient à empiler un élément dans la pile principale. Par contre pour `insérer_gauche(t,e)` il faut dépiler l'ensemble des éléments de la pile principale dans la pile secondaire puis empiler le nouvel élément dans la pile principale vide et enfin dépiler l'ensemble des éléments de la pile secondaire dans la pile principale.

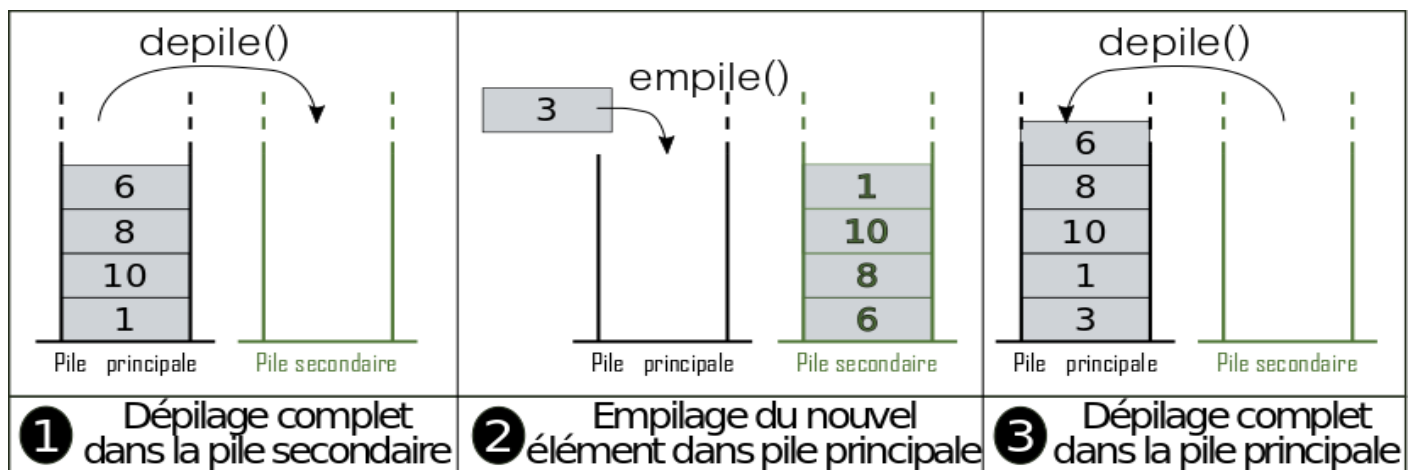


Figure 1: Illustration de la primitive `insérer_gauche(t, e)`

Ces deux primitives ainsi que le constructeur du TAD peuvent s'écrire :

```

fonction creer_2extremities() :
    retourner creer_pile()

fonction insérer_droite(t, e)
    empiler(t, e)

fonction insérer_gauche(t, e) :
    t_secondaire = creer_pile()
    tant que non(est_vide(t)) :
        empiler(t_secondaire, depiler(t))
    empiler(t, e)
    tant que non(est_vide(t_secondaire)) :
        empiler(t, depiler(t_secondaire))
    
```

Q1. Définir les primitives `2_est_vide(t)` et `retirer_droite(t)`

Q2. Illustrer l'évolution des 2 piles (principale et secondaire) lors de l'appel de `retirer_gauche(t)`.

Q3. Définir la primitive `retirer_gauche(t)`



4. Exercice 4 : Implémentation d' une pile avec un tableau

On souhaite définir une classe Pile implémentant une pile à base d'un tableau.

Le tableau utilisé peut contenir n éléments :

- la première case du tableau (indice 0) contient l'indice de la prochaine case vide (emplacement correspondant à l'indice où sera inséré le prochain élément)
- les cases suivantes du tableau (indices 1 à n), contiennent les éléments de la pile ou sont vides. La dernière case non vide correspond au sommet de la pile

D'après cette description, on en déduit que :

- si $P[0] == 1$ → la pile est vide.
 - A chaque fois que l'on insère un élément, on augment $P[0]$ d'une unité
 - Lorsque $P[0] == n+1$, la pile est pleine
- Le constructeur de la classe Pile est la suivante :

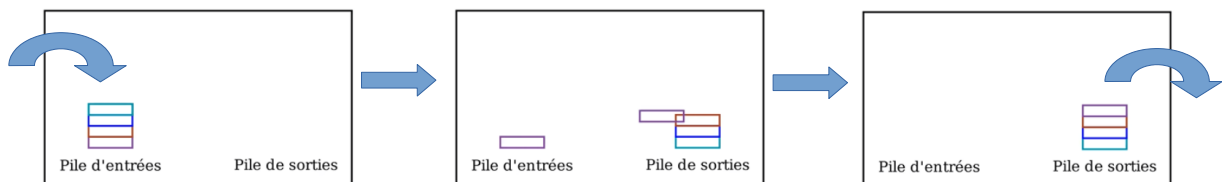
```
1 class Pile :
2     def __init__(self) :
3         self.__n = 20
4         self.__p = [None] * self.__n
5         self.__p[0] = 1
```

Q4. Définir dans cette classe les méthode `empiler()`, `depiler()` et `est_vide()` en respectant l'interface du TAD pile défini dans le cours.

Q5. Valider le fonctionnement de cette classe en vérifiant les résultats obtenus dans l'exercice 2 - séquence 4

5. Exercice 5 : Implémentation d' une file avec deux piles

On souhaite maintenant définir une classe File implémentant une file à base de deux piles. Ces piles seront une une instance de la classe Pile définie dans l'exercice 4.



Principe : on enfila sur la pile d'entrée et on défile sur la pile de sortie

Q6. A partir de la classe Pile et des raisonnements réalisés dans l'exercice 3, **définir** une classe File respectant l'interface d'une file présentée dans le cours.

Q7. Valider le fonctionnement de cette classe en vérifiant les résultats obtenus dans l'exercice 2 - séquence 2

